

Phase 4 Code Challenge: Pizza Restaurants (Updated) (CodeGrade)

 [. \(https://github.com/learn-co-curriculum/python-phase-4-code-challenge-pizza\)](https://github.com/learn-co-curriculum/python-phase-4-code-challenge-pizza)  [. \(https://github.com/learn-co-curriculum/python-phase-4-code-challenge-pizza/issues/new\)](https://github.com/learn-co-curriculum/python-phase-4-code-challenge-pizza/issues/new)

For this assessment, you'll be working with a Pizza Restaurant domain.

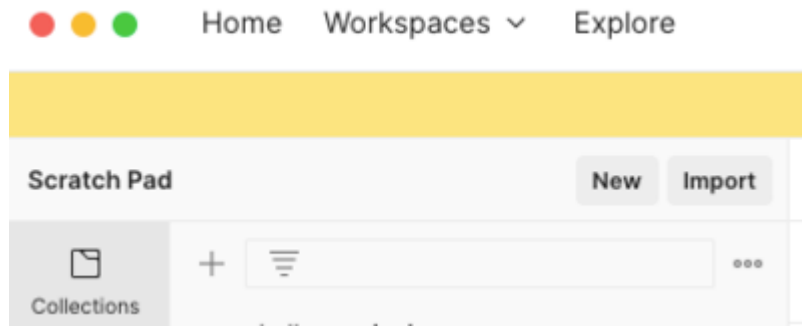
In this repo:

- There is a Flask application with some features built out.
- There is a fully built React frontend application.
- There are tests included which you can run using `pytest -x`.
- There is a file `challenge-1-pizzas.postman_collection.json` that contains a Postman collection of requests for testing each route you will implement.

Depending on your preference, you can either check your API by:

- Using Postman to make requests
- Running `pytest -x` and seeing if your code passes the tests
- Running the React application in the browser and interacting with the API via the frontend

You can import `challenge-1-pizzas.postman_collection.json` into Postman by pressing the `Import` button.



Select **Upload Files**, navigate to this repo folder, and select **challenge-1-pizzas.postman_collection.json** as the file to import.

Setup

The instructions assume you changed into the **code-challenge** folder **prior** to opening the code editor.

To download the dependencies for the frontend and backend, run:

```
pipenv install
pipenv shell
npm install --prefix client
```

You can run your Flask API on **localhost:5555**  (<http://localhost:5555>) by running:

```
python server/app.py
```

You can run your React app on **localhost:4000**  (<http://localhost:4000>) by running:

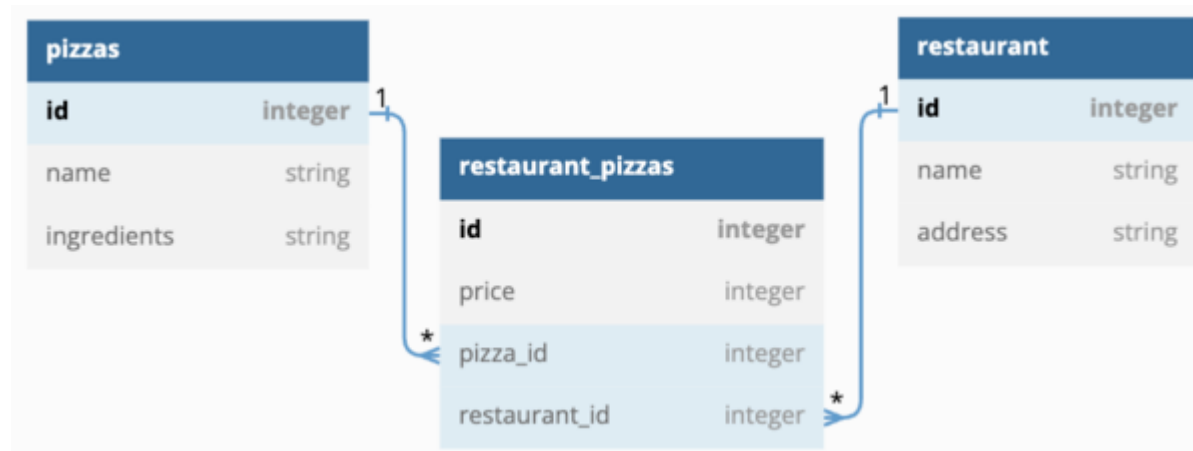
```
npm start --prefix client
```

You are not being assessed on React, and you don't have to update any of the React code; the frontend code is available just so that you can test out the behavior of your API in a realistic setting.

Your job is to build out the Flask API to add the functionality described in the deliverables below.

Models

You will implement an API for the following data model:



The file `server/models.py` defines the model classes **without relationships**. Use the following commands to create the initial database `app.db` :

```
export FLASK_APP=server/app.py
flask db init
flask db migrate
flask db upgrade head
```

Now you can implement the relationships as shown in the ER Diagram:

- A **Restaurant** has many **Pizza** s through **RestaurantPizza**
- A **Pizza** has many **Restaurant** s through **RestaurantPizza**
- A **RestaurantPizza** belongs to a **Restaurant** and belongs to a **Pizza**

Update `server/models.py` to establish the model relationships. Since a **RestaurantPizza** belongs to a **Restaurant** and a **Pizza** , configure the model to cascade deletes.

Set serialization rules to limit the recursion depth.

Run the migrations and seed the database:

```
flask db revision --autogenerate -m 'message'
flask db upgrade head
python server/seed.py
```

If you aren't able to get the provided seed file working, you are welcome to generate your own seed data to test the application.

Validations

Add validations to the `RestaurantPizza` model:

- must have a `price` between 1 and 30

Routes

Set up the following routes. Make sure to return JSON data in the format specified along with the appropriate HTTP verb.

Recall you can specify fields to include or exclude when serializing a model instance to a dictionary using `to_dict()` (don't forget the comma if specifying a single field).

NOTE: If you choose to implement a Flask-RESTful app, you need to add code to instantiate the `Api` class in `server/app.py`.

GET /restaurants

Return JSON data in the format below:

```
[
  {
    "address": "address1",
    "id": 1,
    "name": "Karen's Pizza Shack"
  },
]
```

```
{
  "address": "address2",
  "id": 2,
  "name": "Sanjay's Pizza"
},
{
  "address": "address3",
  "id": 3,
  "name": "Kiki's Pizza"
}
]
```

Recall you can specify fields to include or exclude when serializing a model instance to a dictionary using `to_dict()` (don't forget the comma if specifying a single field).

GET /restaurants/int:id

If the `Restaurant` exists, return JSON data in the format below:

```
{
  "address": "address1",
  "id": 1,
  "name": "Karen's Pizza Shack",
  "restaurant_pizzas": [
    {
      "id": 1,
      "pizza": {
        "id": 1,
        "ingredients": "Dough, Tomato Sauce, Cheese",
        "name": "Emma"
      },
      "pizza_id": 1,
      "price": 1,
    }
  ]
}
```

```
    "restaurant_id": 1
  }
]
```

If the **Restaurant** does not exist, return the following JSON data, along with the appropriate HTTP status code:

```
{
  "error": "Restaurant not found"
}
```

DELETE /restaurants/int:id

If the **Restaurant** exists, it should be removed from the database, along with any **RestaurantPizza** s that are associated with it (a **RestaurantPizza** belongs to a **Restaurant**). If you did not set up your models to cascade deletes, you need to delete associated **RestaurantPizza** s before the **Restaurant** can be deleted.

After deleting the **Restaurant** , return an *empty* response body, along with the appropriate HTTP status code.

If the **Restaurant** does not exist, return the following JSON data, along with the appropriate HTTP status code:

```
{
  "error": "Restaurant not found"
}
```

GET /pizzas

Return JSON data in the format below:

```
[
  {
    "id": 1,
```

```
    "ingredients": "Dough, Tomato Sauce, Cheese",
    "name": "Emma"
  },
  {
    "id": 2,
    "ingredients": "Dough, Tomato Sauce, Cheese, Pepperoni",
    "name": "Geri"
  },
  {
    "id": 3,
    "ingredients": "Dough, Sauce, Ricotta, Red peppers, Mustard",
    "name": "Melanie"
  }
]
```

POST /restaurant_pizzas

This route should create a new **RestaurantPizza** that is associated with an existing **Pizza** and **Restaurant** . It should accept an object with the following properties in the body of the request:

```
{
  "price": 5,
  "pizza_id": 1,
  "restaurant_id": 3
}
```

If the **RestaurantPizza** is created successfully, send back a response with the data related to the **RestaurantPizza** :

```
{
  "id": 4,
  "pizza": {
    "id": 1,
```

```
{
  "ingredients": "Dough, Tomato Sauce, Cheese",
  "name": "Emma"
},
{
  "pizza_id": 1,
  "price": 5,
  "restaurant": {
    "address": "address3",
    "id": 3,
    "name": "Kiki's Pizza"
  },
  "restaurant_id": 3
}
```

If the **RestaurantPizza** is **not** created successfully due to a validation error, return the following JSON data, along with the appropriate HTTP status code:

```
{
  "errors": ["validation errors"]
}
```

This tool needs to be loaded in a new browser window

Load Phase 4 Code Challenge: Pizza Restaurants (Updated) (CodeGrade) in a new window